⏳ Especificación de Requisitos de Software: Proyecto Kronos (V0.2 Simplificada)

1. Arquitectura de Infraestructura y Entorno

El sistema está completamente contenerizado para garantizar entornos reproducibles independientes de la máquina local.

- **Motor de Ejecución:** Python 3.12-slim en un contenedor Docker con flujos de salida desbufferizados (PYTHONUNBUFFERED=1).
- **Persistencia de Datos:** Volumen nominado de Docker (db_data) montado en /data/ para persistir el archivo SQLite.
- **Control de Cambios del Esquema:** Migraciones automatizadas mediante Alembic para gestionar la evolución del modelo relacional.

2. Modelo de Datos Relacional (Esquema SQLite)

Se implementa una arquitectura con restricciones de claves foráneas activas (PRAGMA foreign_keys = ON;).

Tabla: habits (Definición Base de Hábitos)

Campo	Tipo	Restricciones	Descripción
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador único del hábito.
name	TEXT	NOT NULL UNIQUE	Nombre del hábito (ej. "Pushups").
amount	REAL	NOT NULL DEFAULT 1.0	Meta cuantitativa diaria (1.0 para binarios).
unit	TEXT	NOT NULL DEFAULT 'check'	Unidad física (ej. "Reps", o "check" para binarios).

status	TEXT	NOT NULL DEFAULT 'ACTIVE'	Estados: ACTIVE, PAUSED, ARCHIVED.
current_xp	INTEGER	NOT NULL DEFAULT 0	Experiencia acumulada total.
level	INTEGER	NOT NULL DEFAULT 1	Nivel actual del hábito (Mecánica RPG).

Tabla: habit_logs (Historial de Ejecución)

Campo	Tipo	Restricciones	Descripción
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador del registro.
habit_id	INTEGER	NOT NULL, FOREIGN KEY	Enlace a habits(id) ON DELETE CASCADE.
logged_date	TEXT	NOT NULL	Fecha en formato estricto ISO YYYY-MM-DD.
amount_done	REAL	NOT NULL	Cantidad real efectuada por el usuario.
target_snapshot	REAL	NOT NULL	Copia inmutable del amount del blueprint ese día.

unit_snapshot	TEXT	NOT NULL	Copia inmutable de la unit del hábito ese día.

- **Restricción de Integridad:** Clave única compuesta por UNIQUE(habit_id, logged_date) para impedir duplicidad de registros en el mismo día.

3. Especificación de Funcionalidades Nucleares (API REST)

RF1: Gestión de Hábitos (Blueprints)

- Permite registrar hábitos con nombres exclusivos.
- Soporta edición de name, amount y unit mediante verbos HTTP PUT/PATCH. Si el nuevo name ya existe en otro hábito, la API debe capturar el error de unicidad y devolver un código HTTP 400.

RF2: Tablero Diario de Control (Checklist)

- **Query de Aislamiento Diario:** Para generar el checklist de una fecha específica (:target_date), el sistema ejecuta un LEFT JOIN aplicando el filtro de fecha directamente en la condición de unión:

```
SQL
SELECT h.*, l.amount_done, l.target_snapshot, l.unit_snapshot
FROM habits h
LEFT JOIN habit_logs l ON h.id = l.habit_id AND l.logged_date = :target_date
WHERE h.status = 'ACTIVE';
```

- **Cálculo de Progreso:** Se evalúa el ratio entre amount_done y el objetivo (target_snapshot si existe el log, o amount base si no existe). El frontend mapea los estados: 0% (Sin registro), <100% (En progreso), ≥100% (Completado).

RF3: Registro Idempotente de Progreso

La API expone dos endpoints independientes y predecibles:

- POST /habits/{id}/logs: Recibe un payload JSON con logged_date y amount_done. Si no se envía amount_done, el backend hereda automáticamente el amount del blueprint. Si el registro ya existe para esa fecha, realiza un **UPSERT** (actualiza el valor existente y recalcula XP).
- DELETE /habits/{id}/logs/{logged_date}: Elimina explícitamente el registro de ese día de forma limpia (Unmark) y resta la XP correspondiente.

RF4: Mecanismo de Snapshotting de Metas

Al consolidar cualquier registro diario (POST), se inyectan obligatoriamente los campos target_snapshot y unit_snapshot. Cualquier mutación futura en las metas de la tabla habits no alterará los porcentajes de éxito del pasado.

RF5: Control de Ciclo de Vida (Soft-State Archiving)

Se prohíbe el borrado físico de la tabla habits:

- ACTIVE: Fluye al checklist diario.
- PAUSED: No aparece en el checklist ni permite registrar progreso, pero preserva la XP y el histórico.
- ARCHIVED: Almacenamiento en frío. Desaparece de las vistas comunes.

4. Reglas de Gamificación y Progreso Matemático

Mecánica de Niveles RPG y Curva Lineal

- **Asignación de XP:** Cada registro diario que alcance o supere el 100% del objetivo otorga una constante fija de **10 Puntos de Experiencia (XP)**. Los progresos parciales (<100%) otorgan XP proporcional de forma lineal:
  $$\text{XP Ganada} = \left(\frac{\text{amount_done}}{\text{target_snapshot}}\right) \times 10$$
- **Ecuación de Progreso:** La experiencia requerida para subir al siguiente nivel se calcula de manera directa sobre el nivel actual del hábito:
  $$\text{XP Requerida para Nivel Siguiente} = \text{level} \times 100$$
- **Sincronización Bidireccional:** El nivel y la XP son puramente matemáticos. Si el usuario realiza un POST (añade/actualiza progreso), la XP sube y el nivel se calcula hacia arriba. Si el usuario realiza un DELETE (elimina progreso), la XP baja y, si cae por debajo del umbral necesario, el nivel disminuye en consecuencia.

5. Diseño de Contratos de la API (Pydantic / FastAPI)

Los endpoints generan documentación interactiva estandarizada bajo OpenAPI (/docs).

Python
```
from datetime import date
from typing import Optional
from pydantic import BaseModel, Field

class HabitCreate(BaseModel):
    name: str = Field(..., min_length=1, max_length=50)
    amount: float = Field(default=1.0, gt=0)
    unit: str = Field(default="check", min_length=1)

class LogCreate(BaseModel):
    logged_date: date  # Pydantic valida formato YYYY-MM-DD; el backend lo guarda como string en SQLite
    amount_done: Optional[float] = None  # Si es None, hereda el amount del blueprint
```